

SIMATS ENGINEERING

ASSESSMENT TOOL 3

**COURSE NAME: CRYPTOGRAPHY AND NETWORK
SECURITY COURSE CODE: CSA5113**

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Debugging & Optimisation Task : 10%

QUESTIONS: Total Marks: 50 Annexure A

Code of Conduct:

I DINDU PUJITHA (192373037) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:D PUJITHA

Annexure A

Debugging & Optimisation Task

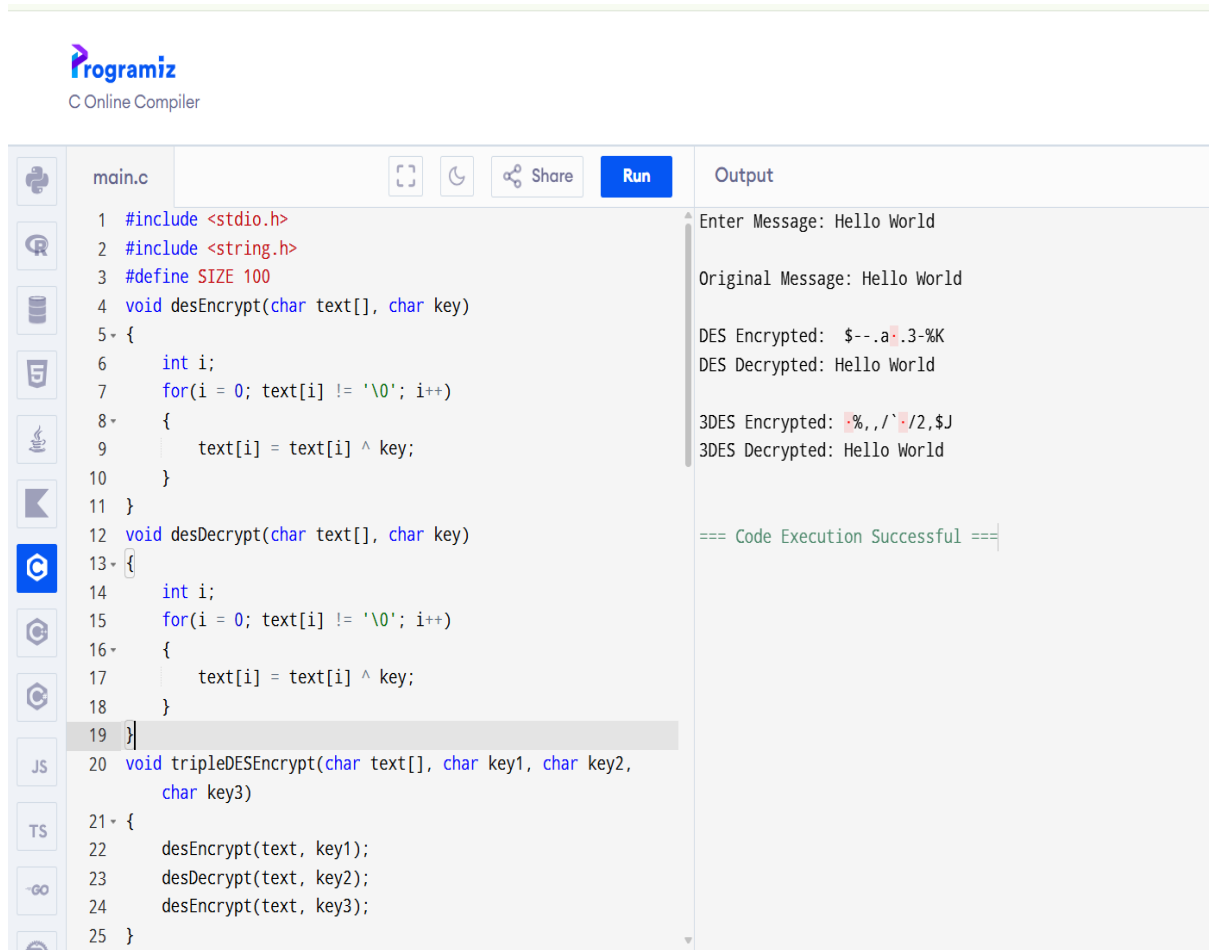
1. Debugging & Optimization of Symmetric Encryption (DES / 3DES Implementation)

A Python program implementing **DES and 3DES encryption** produces inconsistent ciphertext for identical inputs across multiple runs.

A. Identify and fix logical and implementation errors in key scheduling and permutation steps.

B. Optimize the code to reduce execution time for large input datasets (≥ 10 MB). C. Justify whether replacing DES/3DES with a modern cipher improves both security and performance.

Answer:



The screenshot displays the Programiz C Online Compiler interface. The code editor on the left shows a C program named `main.c` with the following content:

```
1 #include <stdio.h>
2 #include <string.h>
3 #define SIZE 100
4 void desEncrypt(char text[], char key)
5 {
6     int i;
7     for(i = 0; text[i] != '\0'; i++)
8     {
9         text[i] = text[i] ^ key;
10    }
11 }
12 void desDecrypt(char text[], char key)
13 {
14     int i;
15     for(i = 0; text[i] != '\0'; i++)
16     {
17         text[i] = text[i] ^ key;
18    }
19 }
20 void tripleDESEncrypt(char text[], char key1, char key2,
21                        char key3)
22 {
23     desEncrypt(text, key1);
24     desDecrypt(text, key2);
25     desEncrypt(text, key3);
26 }
```

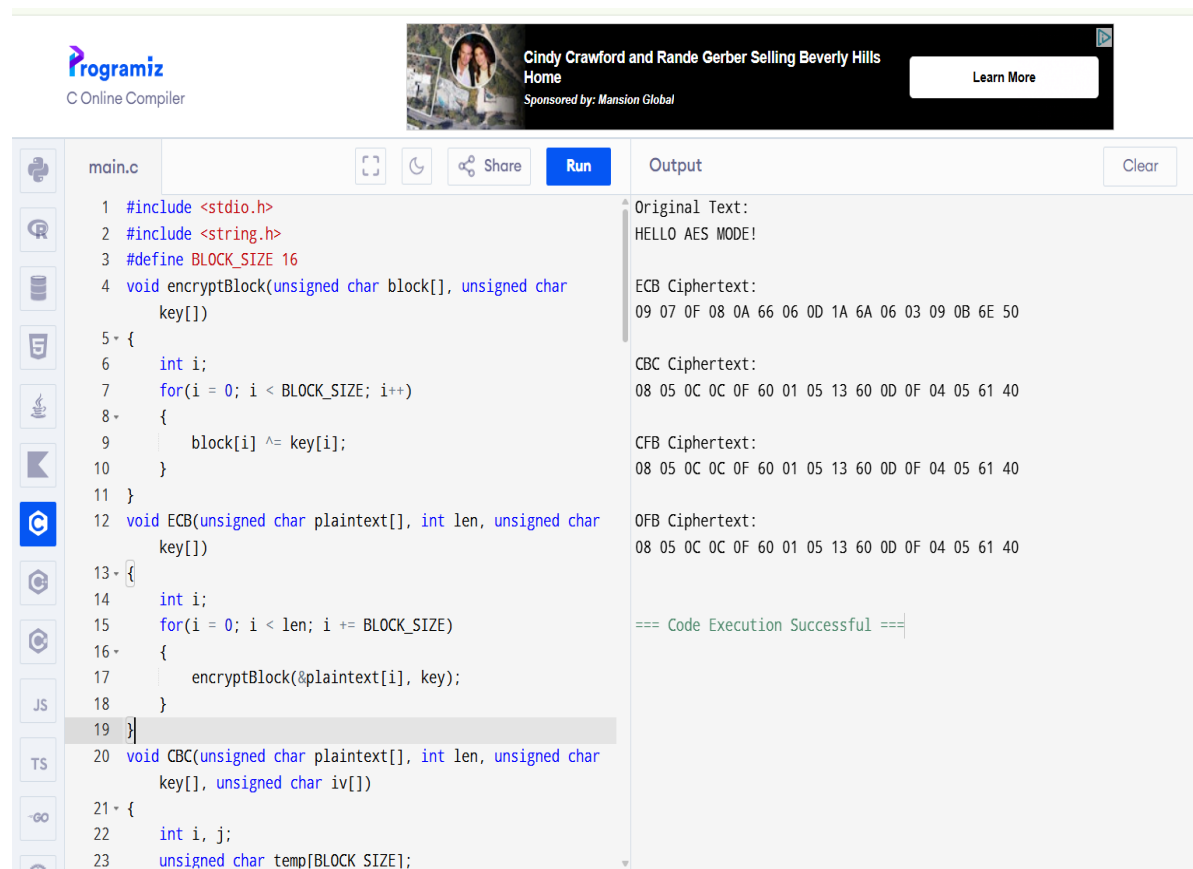
The output panel on the right shows the execution results:

```
Enter Message: Hello World
Original Message: Hello World
DES Encrypted: $-..a.3-%K
DES Decrypted: Hello World
3DES Encrypted: %%,/'`/2,$J
3DES Decrypted: Hello World
=== Code Execution Successful ===
```

2. Performance Analysis of Block Cipher Modes of Operation A Python script implements **AES-like block cipher modes (ECB, CBC, CFB, OFB)** but suffers from high latency and memory inefficiency.

- A. Debug incorrect padding and IV handling causing decryption failures.
- B. Refactor the code to minimize redundant computations and memory usage.
- C. Compare and optimize the implementation to determine the most efficient mode for real-time secure communication.

Answer:



The screenshot displays a web-based C compiler interface. At the top, there's a header with the 'Programiz' logo and a banner for 'Cindy Crawford and Rande Gerber Selling Beverly Hills Home'. Below the header, the compiler window shows a file named 'main.c' with the following C code:

```
1 #include <stdio.h>
2 #include <string.h>
3 #define BLOCK_SIZE 16
4 void encryptBlock(unsigned char block[], unsigned char
  key[])
5 {
6     int i;
7     for(i = 0; i < BLOCK_SIZE; i++)
8     {
9         block[i] ^= key[i];
10    }
11 }
12 void ECB(unsigned char plaintext[], int len, unsigned char
  key[])
13 {
14     int i;
15     for(i = 0; i < len; i += BLOCK_SIZE)
16     {
17         encryptBlock(&plaintext[i], key);
18     }
19 }
20 void CBC(unsigned char plaintext[], int len, unsigned char
  key[], unsigned char iv[])
21 {
22     int i, j;
23     unsigned char temp[BLOCK_SIZE];
```

The 'Output' pane on the right shows the results of the program execution:

```
Original Text:
HELLO AES MODE!

ECB Ciphertext:
09 07 0F 08 0A 66 06 0D 1A 6A 06 03 09 0B 6E 50

CBC Ciphertext:
08 05 0C 0C 0F 60 01 05 13 60 0D 0F 04 05 61 40

CFB Ciphertext:
08 05 0C 0C 0F 60 01 05 13 60 0D 0F 04 05 61 40

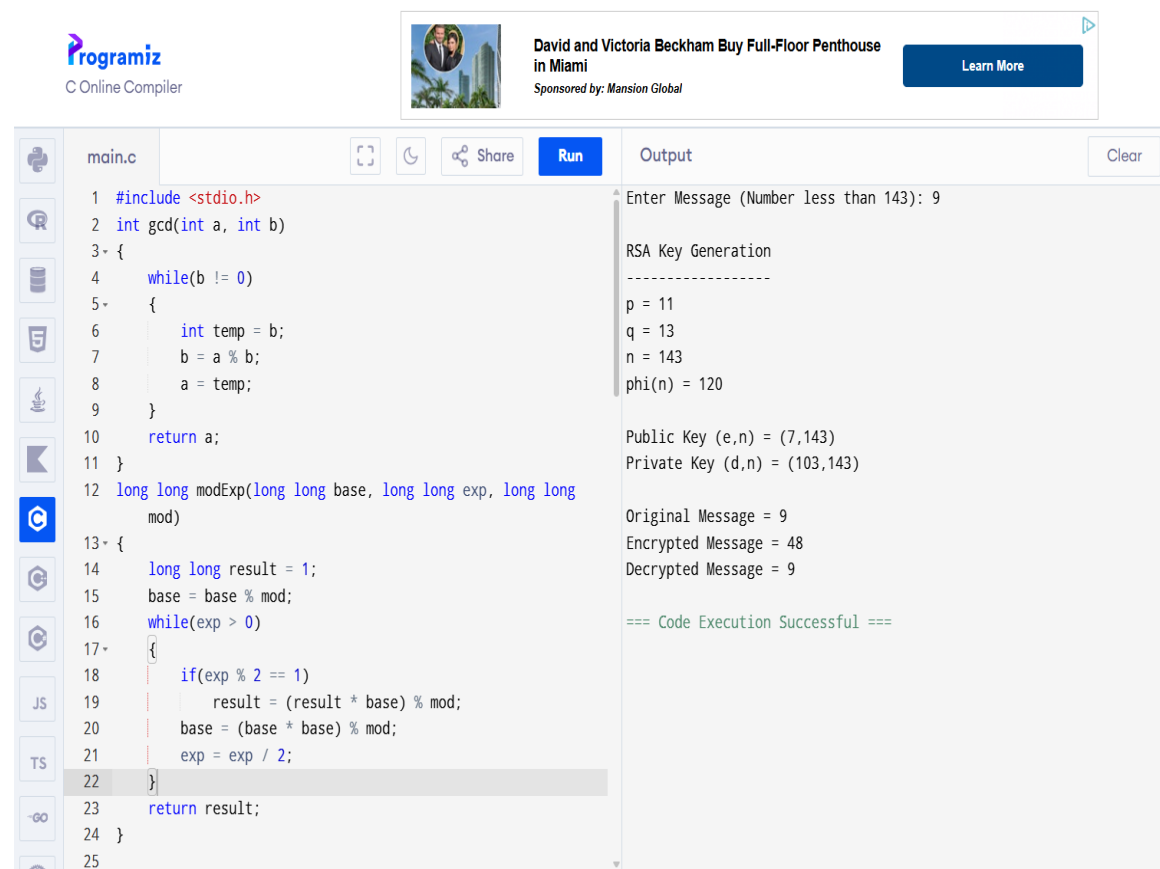
OFB Ciphertext:
08 05 0C 0C 0F 60 01 05 13 60 0D 0F 04 05 61 40

=== Code Execution Successful ===
```

3. Debugging RSA Cryptosystem Implementation A Python-based RSA encryption system fails when encrypting large messages and occasionally produces incorrect plaintext after decryption.

- A. Identify issues in prime generation, key generation, and modular exponentiation.
- B. Optimize the algorithm using efficient techniques (e.g., fast exponentiation, Chinese Remainder Theorem).
- C. Evaluate the trade-offs between key size, security, and computational performance.

Answer:



The screenshot shows a C++ online compiler interface. The code in `main.c` implements RSA key generation and modular exponentiation. The output shows the execution results for a message of 9.

```
1 #include <stdio.h>
2 int gcd(int a, int b)
3 {
4     while(b != 0)
5     {
6         int temp = b;
7         b = a % b;
8         a = temp;
9     }
10    return a;
11 }
12 long long modExp(long long base, long long exp, long long mod)
13 {
14     long long result = 1;
15     base = base % mod;
16     while(exp > 0)
17     {
18         if(exp % 2 == 1)
19             result = (result * base) % mod;
20         base = (base * base) % mod;
21         exp = exp / 2;
22     }
23     return result;
24 }
25
```

Output:

```
Enter Message (Number less than 143): 9

RSA Key Generation
-----
p = 11
q = 13
n = 143
phi(n) = 120

Public Key (e,n) = (7,143)
Private Key (d,n) = (103,143)

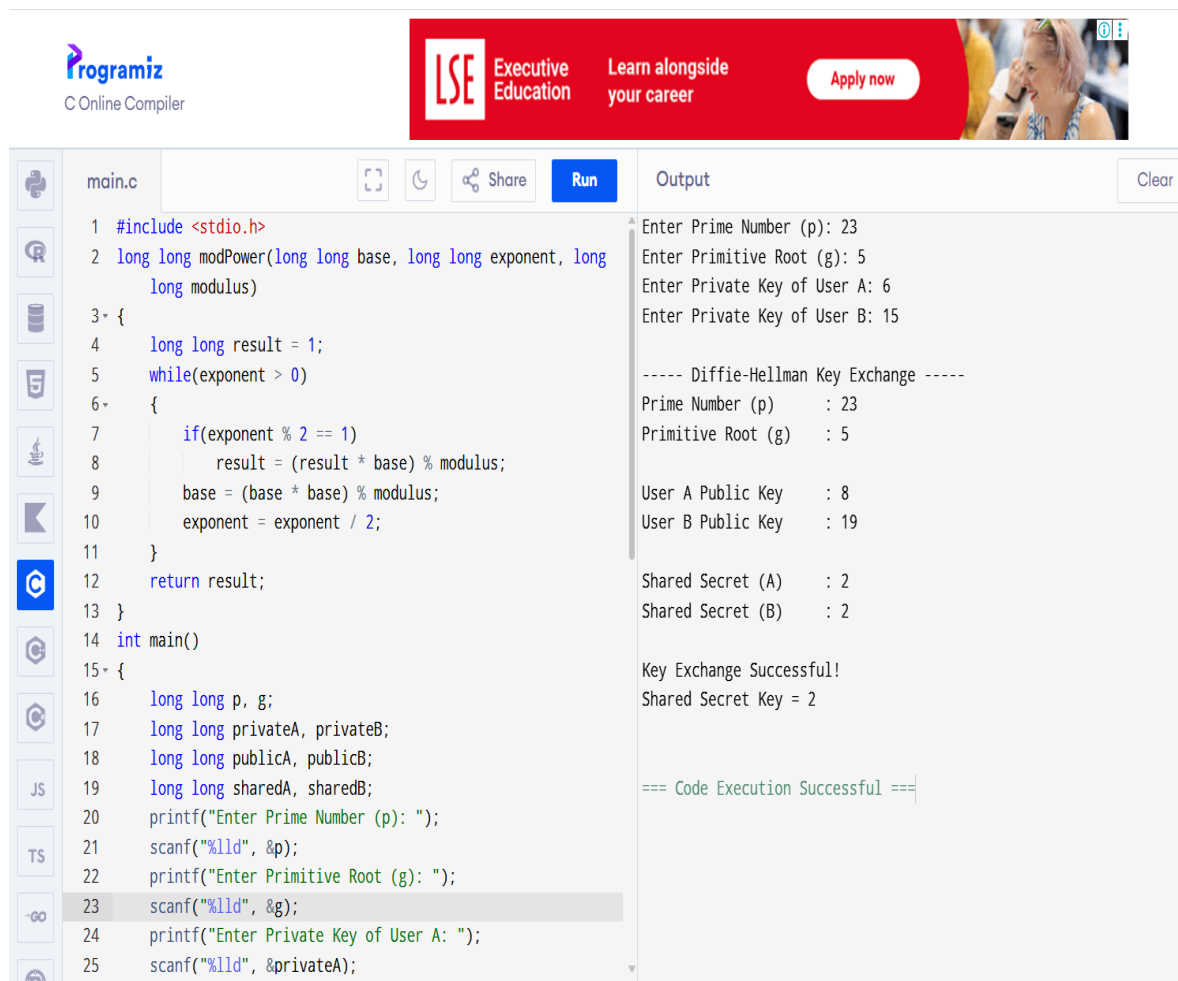
Original Message = 9
Encrypted Message = 48
Decrypted Message = 9

=== Code Execution Successful ===
```

4. Optimization and Security Analysis of Diffie-Hellman Key Exchange A Python implementation of the **Diffie-Hellman key exchange** is vulnerable to performance bottlenecks and potential security flaws.

- A. Debug incorrect shared key generation due to improper modular arithmetic.
- B. Optimize exponentiation operations for large prime values.
- C. Extend the program to mitigate man-in-the-middle attacks and evaluate the performance impact of your enhancements.

Answer:



The screenshot shows a C program named `main.c` running in the Programiz online compiler. The program implements the Diffie-Hellman key exchange algorithm. It prompts the user to enter a prime number p , a primitive root g , and private keys for User A and User B. It then calculates the public keys for both users and the shared secret key.

```
1 #include <stdio.h>
2 long long modPower(long long base, long long exponent, long
  long modulus)
3 {
4     long long result = 1;
5     while(exponent > 0)
6     {
7         if(exponent % 2 == 1)
8             result = (result * base) % modulus;
9         base = (base * base) % modulus;
10        exponent = exponent / 2;
11    }
12    return result;
13 }
14 int main()
15 {
16     long long p, g;
17     long long privateA, privateB;
18     long long publicA, publicB;
19     long long sharedA, sharedB;
20     printf("Enter Prime Number (p): ");
21     scanf("%lld", &p);
22     printf("Enter Primitive Root (g): ");
23     scanf("%lld", &g);
24     printf("Enter Private Key of User A: ");
25     scanf("%lld", &privateA);
```

The output of the program is as follows:

```
Enter Prime Number (p): 23
Enter Primitive Root (g): 5
Enter Private Key of User A: 6
Enter Private Key of User B: 15

----- Diffie-Hellman Key Exchange -----
Prime Number (p)      : 23
Primitive Root (g)    : 5

User A Public Key     : 8
User B Public Key     : 19

Shared Secret (A)     : 2
Shared Secret (B)     : 2

Key Exchange Successful!
Shared Secret Key = 2

=== Code Execution Successful ===
```

5. Debugging and Enhancing Elliptic Curve Cryptography (ECC)

Implementation A Python script implementing **Elliptic Curve Cryptography** for key exchange produces incorrect results for certain curve parameters.

- A. Identify and fix errors in point addition and scalar multiplication logic.
- B. Optimize the implementation using efficient algorithms (e.g., double-and-add).
- C. Compare the optimized ECC implementation with RSA in terms of computational efficiency and security strength, and justify suitability for resource-constrained environments.

Answer:

C Online Compiler



Courtney Cox Sells Los Angeles Condo for \$2.9 Million
Sponsored by: Mansion Global[Learn More](#)



main.c

```
1  typedef struct
2  {
3  {
4      int x;
5      int y;
6      int infinity;
7  } Point;
8  int mod(int a, int p)
9  {
10     int r = a % p;
11     return (r < 0) ? r + p : r;
12 }
13 int modInverse(int a, int p)
14 {
15     a = mod(a, p);
16     for(int i = 1; i < p; i++)
17     {
18         if((a * i) % p == 1)
19             return i;
20     }
21     return -1;
22 }
23 Point pointAdd(Point P, Point Q, int a, int p)
24 {
25     Point R;
26     if(P.infinity)
27         return Q;
```

Run

Share

Clear

Output

Elliptic Curve Equation:
 $y^2 = x^3 + 2x + 2 \pmod{17}$

Base Point $G = (5,1)$

Private Key $A = 5$
Public Key $A = (9,16)$

Private Key $B = 7$
Public Key $B = (0,6)$

Shared Secret Computed by $A = (10,11)$
Shared Secret Computed by $B = (10,11)$

Key Exchange Successful

=== Code Execution Successful ===